



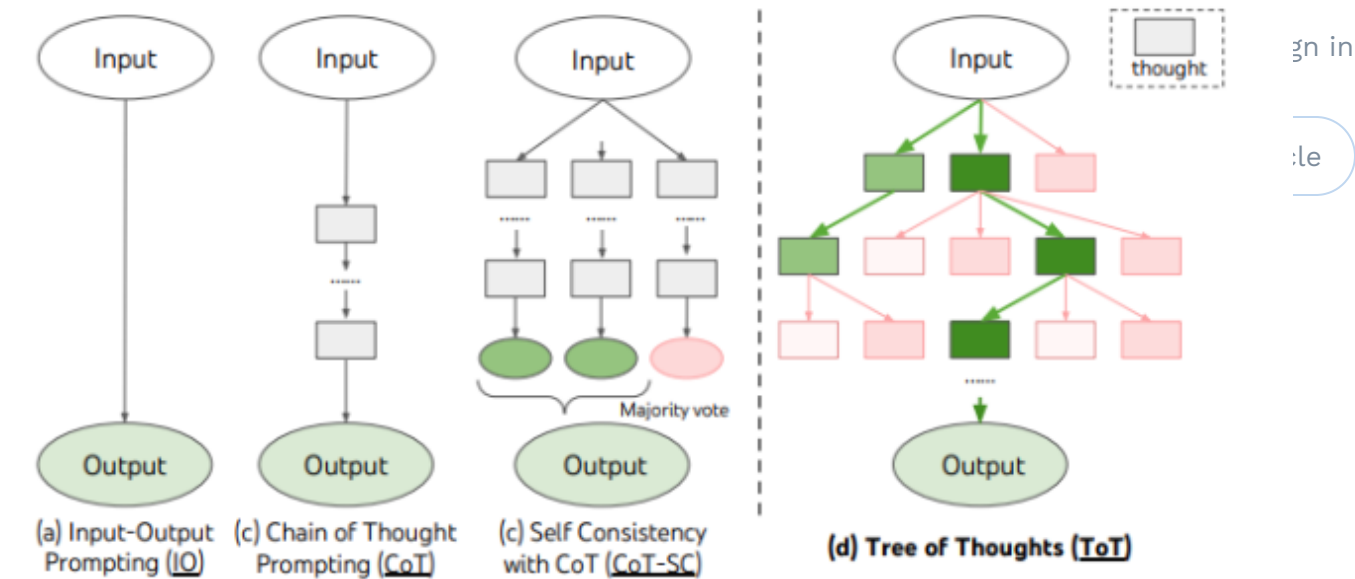
LARGE LANGUAGE MODELS

Prompt Engineering for Arithmetic Reasoning Problems

Explore various prompt engineering techniques for arithmetic reasoning problems, best practices, and rapid experimentations

Kaustubh Bhavsar
Nov 18, 2023 16 min read

- LATEST
- EDITOR'S PICKS
- DEEP DIVES
- NEWSLETTER



Architecture of four different prompting techniques: Input-Output, Chain-of-Thought (CoT), Self Consistency with Chain-of-Thought (CoT), and Tree of Thought (ToT) (Image from Yao et al. (2023))

Introduction

Large Language Models (LLMs) have been drawing attention from both academic researchers and industry practitioners.

due to their proficiency in understanding and generating language. The reason behind their text comprehension skills lies in their training process, which involves exposure to vast amounts of data with the primary goal of predicting subsequent words. To optimize these models for specific tasks, fine-tuning is essential. This can be achieved through two methods: 'pre-training and fine-tuning' or 'prompt fine-tuning'.

In the conventional 'pretraining and fine-tuning' LLM is fine-tuned on a dataset pertinent to the perform later, thereby updating parameters in the phase. Conversely, 'prompt fine-tuning' guides the text segment to execute the task.

A prompt is an input provided by the user that is designed to respond to. Prompts can contain instructions, a question or an output indicator. Prompt engineering is an emerging field dedicated to the development and refinement of prompts for the effective utilization of language models.

However, a significant challenge lies in ensuring the model can tackle problems necessitating arithmetic and common sense reasoning. In this article, our focus is specifically on the engineering of prompts for arithmetic problems.

Prerequisites

No prior knowledge is required. All the demonstrations below can be executed either on the [OpenAI Playground](#) through the [OpenAI API](#). Although this article primarily focuses on OpenAI models, it's important to note that these concepts are applicable to other LLMs as well.

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

prompting techniques that we'll be exploring, and you're free to apply any LLM available in the market.

Prompt Engineering for Arithmetic Problems

All the problems mentioned henceforth are taken from the GSM8K dataset and are tested using OpenAI's **GPT-4o** **Instruct** model with default configurations.

We will be testing various techniques specifically on an arithmetic problem:

Jennifer purchased 40 cans of milk at the store before her classmate Mark, who was also buying milk. Jennifer bought 2 cans for every 5 cans Mark bought. If Mark purchased 50 cans of milk, how many cans of milk did Jennifer bring home from the store?

The correct answer to the above problem is **100**.

Zero-Shot Prompting

A **shot** essentially refers to **an example**. Zero-shot prompting technique in which a question is posed without providing any demonstrations. This technique produces favourable results in larger models that are trained on extensive amounts of data.

Prompt:

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

Q: {{ question }}

A:

Output:

Jennifer brought home 76 cans of milk from the store.

In the preceding prompt, we didn't prepend the question with any demonstrations, leading to the models' failure to provide a correct answer for the arithmetic problem. Real-world commonsense problems seldom yield satisfactory results from zero-shot prompts. Therefore, it's essential to include demonstrations when dealing with such problems.

Few-Shot Prompting

With few-shot prompting, we provide the model with a small number of demonstrations, also known as exemplars. These exemplars serve to guide the model so that it can handle similar questions. The number of demonstrations needed varies based on the complexity of the problem and the specific model in use.

Prompt:

Q: Ken created a care package to send to his brother,

A: 16

Q: Mark has a garden with flowers. He planted plants in

A: 35

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

Q: James writes a 3-page letter to 2 different friends twice a week. How many

A: 624

Q: Albert is wondering how much pizza he can eat in one day. He buys 2 large

A: 48

Q: Betty is saving money for a new wallet which costs \$100. Betty has only 1

A: 5

Q: {{ question }}

A:

Output:

76

Although this technique works well in tackling d the model may still fail to find satisfactory resul reasoning problems. This is because the model immediate steps.

Few-Shot Chain-of-Thought (CoT) I

Few-shot chain-of-thought (CoT) prompting equ with a few examples to guide it through the pro solution. This guidance, also known as intermed steps, is crucial in assisting the model to work t and generate the desired output logically.

Prompt:

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

Q: James writes a 3-page letter to 2 different friends twice a week. How many

A: He writes each friend $3 \times 2 = 6$ pages a week So he writes $6 \times 2 = 12$ pages every

Q: {{ question }}

A:

Output:

Jennifer bought $40 + 6/5 \times 50 = 40 + 12 \times 10 = 40 + 120 = 160$ cans of

In the above output, the model attempted to restate the problem in its own words, but arrived at the incorrect final answer. This discrepancy may be attributed to the irrelevant information provided. There are two ways to address this issue: provide the model with more examples or ensure that the information provided is relevant. Below, we attempt to rectify this with a new example.

Prompt:

Q: Mark has a garden with flowers. He planted plants in the garden.

A: There are $80/100 \times 10 = 8$ more purple flowers than yellow flowers.

Q: {{ question }}

A:

Output:

For every 5 cans Mark bought, Jennifer bought 6 additional cans.

The final answer obtained is now correct. Consider the model's choice of the model, and the relevance of examples provided.

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

contribute to arriving at the correct solution.

Program-Aided Language Model (PAL)

A program-aided language model (PAL) is not a prompting technique; instead, it is an application of prompt engineering. Similar to CoT, PAL aims to reason out the intermediate steps. However, instead of using regular text as reason uses intermediate programmatic steps using pr runtime such as Python interpreter.

[LATEST](#)
[EDITOR'S PICKS](#)
[DEEP DIVES](#)
[NEWSLETTER](#)
[WRITE FOR TDS](#)
[Sign in](#)
[Submit an Article](#)

Prompt:

Q: James writes a 3-page letter to 2 different friends:

A: #James has

```
number_of_friends=2
```

```
#He writes each friend 3-page letter each time.
```

```
page_letter_size=3
```

```
#He writes twice to each friend in a week.
```

```
number_of_times_written_to_friend=2
```

```
#Total pages written in a week to each friend
```

```
pages_written_to_each_friend_per_week=page_letter_size
```

```
#So, total pages written by him per week,
```

```
total_pages_each_week=pages_written_to_each_friend_per
```

```
#Number of weeks in a year
```

```
total_weeks_in_a_year=52
```

```
#That means he writes,
```

```
total_pages_written_in_year=total_pages_each_week*total_weeks_in_a_year=624
```

```
#pages per year.
```

The answer is 624.

Q: {{ question }}

A:

Output:

```
#Jennifer initially bought 40 cans of milk.
```

```
initial_cans_bought=40
```

```
#For every 5 cans Mark bought, Jennifer bought 6 addi
```

```
additional_cans_bought=6
```

```
mark_cans_bought=50
```

```
#So, for every 5 cans Mark bought, Jennifer bought 6 :
```

```
additional_cans_bought=6
```

```
mark_cans_bought=50
```

```
#Number of times Mark bought 5 cans
```

```
number_of_times_mark_bought=mark_cans_bought/5=10
```

```
#Total additional cans bought by Jennifer
```

```
total_additional_cans_bought=additional_cans_bought*n
```

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

#Total cans of milk brought home by Jennifer

total_cans_brought_home=initial_cans_bought+total_additional_cans_bought=40+

The answer is 100 cans of milk.

Even with the irrelevant example that failed to yield a correct answer using the few-shot CoT technique, PAL successfully found the correct final solution.

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

Zero-Shot Chain-of-Thought (CoT) Prompting

In a situation where demonstrations for the few-shot chain-of-thought (CoT) technique are not available, a straightforward and effective approach is to ask the model itself to generate the intermediate steps. This is called the zero-shot chain-of-thought (CoT). You can achieve this by adding a prompt like 'Let's think step by step...' to the prompt question. This technique has shown exceptional results.

Prompt:

Q: {{ question }}

A: Let's think step by step...

Output:

1. Jennifer initially bought 40 cans of milk.
2. For every 5 cans Mark bought, Jennifer bought 6 additional cans.
3. Since Mark bought 50 cans, Jennifer bought 6 additional cans.
4. Therefore, Jennifer brought home a total of 40 + 60 = 100 cans of milk.

From the above example, we observe that the model successfully reasons through the steps and arrives at the correct solution without requiring any examples or guidance.

Tree-of-Thought (ToT) Prompting

Tree-of-thought (ToT) framework involves exploring potential solutions in a manner akin to navigating a tree of thoughts, similar to human problem-solving. This enables the possibility of retracing steps when necessary, the way humans may reassess and adjust their approach during the problem-solving process. In essence, ToT air the adaptive and iterative nature of human reasoning through trial and error.

Based on the ToT framework, Dave Hulbert proposed thought prompting. This technique employs a structured single-prompt approach, asking the model to ask questions at each step.

Prompt:

Identify and behave as three different experts that are helpful, honest, and concise. All experts will write down the step and their thinking. Then, all experts will go on to the next step, etc. At each step all experts will score their peers responses. If any expert is judged to be wrong at any point then they will be removed. After all experts have provided their analysis, you then

Following is the question:

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

Q: {{ question }}

A:

Output:

Expert 1: Math Expert

Step 1: Calculate the total number of cans of milk purchased by Mark

Mark bought 50 cans of milk.

Score: 5 (highly likely)

Step 2: Calculate the number of cans of milk purchased

Jennifer purchased 40 cans of milk.

Score: 5 (highly likely)

Step 3: Calculate the number of additional cans of milk

For every 5 cans Mark bought, Jennifer bought 6 additional

Score: 5 (highly likely)

Step 4: Calculate the total number of cans of milk purchased

Jennifer bought $40 + (6/5)*50 = 40 + 60 = 100$ cans of

Score: 5 (highly likely)

Step 5: Conclusion

Jennifer brought home 100 cans of milk from the store.

Score: 5 (highly likely)

Expert 2: Retail Expert

Step 1: Determine the average number of cans of milk purchased

Based on previous sales data, the average number of cans

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

Score: 4 (likely)

Step 2: Calculate the total number of cans of milk purchased by Mark.

Mark bought 50 cans of milk.

Score: 5 (highly likely)

Step 3: Calculate the number of cans of milk purchased by Jennifer before me
Jennifer purchased 40 cans of milk.

Score: 5 (highly likely)

LATEST

EDITOR'S PICKS

Step 4: Calculate the total number of cans of milk pur

Jennifer bought $40 + (6/5) * 50 = 40 + 60 = 100$ cans of

Score: 5 (highly likely)

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Step 5: Conclusion

Jennifer brought home 100 cans of milk from the store.

Score: 5 (highly likely)

Sign in

Submit an Article

Expert 3: Psychology Expert

Step 1: Analyze Jennifer's behavior and decision-making

Jennifer may have purchased 40 cans of milk based on h

Score: 4 (likely)

Step 2: Analyze Mark's behavior and decision-making pr

Mark may have purchased 50 cans of milk based on his c

Score: 4 (likely)

Step 3: Consider the influence of social norms and pee

Jennifer may have felt pressured to buy more cans of m

Score: 3 (somewhat likely)

Step 4: Analyze the impact of Jennifer's decision to buy additional cans for Jennifer may have made this decision based on a desire to keep up with Mark
Score: 3 (somewhat likely)

Step 5: Conclusion

Jennifer brought home 100 cans of milk from the store, possibly influenced b
Score: 3 (somewhat likely)

Consensus Solution:

Based on the analysis of all three experts, it is high

The ToT technique employed in the above example is an accurate solution. It's worth mentioning that the solution is taken from Dave Hulbert's ToT prompts. However, a drawback of this technique is a significant increase in size.

Besides the engineering techniques mentioned above, several additional methods can be adopted for solving arithmetic reasoning problems. However, it's important to note that the following techniques – **Automatic Chain-of-Thought (AutoCoT)**, **Consistency**, **Automatic Reasoning and Tool-Use**, and **Active Prompt** – are beyond the scope of this article.

Best Practices

Models perform effectively when fine-tuned with specific prompts. Elvis Saravia noted that a prompt may contain several elements, though not all are necessary each time.

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

1. **Instruction:** This specifies a particular task or action you want the model to undertake.
2. **Context:** External information or additional details that guide the model to generate more accurate responses.
3. **Input Data:** The specific input or question for which we seek a response.
4. **Output Indicator:** The expected type or format of the output.

It is advisable to place the instructions at the beginning and separated from the rest of the prompt using designated separators like ‘#.’

Santu and Feng (2023) introduced a comprehensive framework, TELeR (Turn, Expression, Level of Details, Role), to enable more meaningful comparisons among models. Their taxonomy categorizes prompt directives into six levels based on the level of detail provided. The sixth level can be utilized to optimize the model’s performance based on the prompt:

1. Provide a brief description of the high-level task.
2. Present a detailed bulleted list of sub-tasks.
3. Specify the criteria for evaluating the LLM output or include a few shot examples.
4. Include additional pertinent information obtained from retrieval-based techniques.
5. Request an explicit statement from the LLM regarding the output.

[LATEST](#)[EDITOR’S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

In general, it's advisable to be specific, offer clear details, continuously refine the prompt through iteration, properly format the prompt with clear separations, and mention do's and not don'ts.

Let's revisit the prompt, considering the above-mentioned practices and applying the zero-shot prompting technique.

Prompt:

INSTRUCTION

You are required to answer Arithmetic Reasoning questions.

1. All problems should be solved in as many steps as required.
2. All calculations should be clear and simple to understand.
3. Provide an accurate final answer at the end of your response.

You'll be evaluated on:

- how easier are the calculations done to understand?
- how accurate is the answer?

INPUT DATA

Q: {{ question }}

A:

Output:

1. Jennifer bought 40 cans of milk at the store.
2. Mark bought 50 cans of milk.
3. Jennifer bought 6 additional cans for every 5 cans Mark bought.
4. For every 5 cans Mark bought, Jennifer bought 6 additional cans.

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

5. Therefore, for 50 cans Mark bought, Jennifer bought $(6/5)*50 = 60$ additic

6. Total cans of milk Jennifer bought = $40 + 60 = 100$ cans.

Answer is 100 cans.

Initially, the model failed to provide the correct solution using the zero-shot prompting technique. However, with the application of appropriate instructions and formatting, the same model successfully generates the correct response using the zero-shot prompting technique.

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)

Disclaimer: The following section provides information about Vellum, a developer platform useful for the rapid productionization of prompts. The mention of Vellum is for informational purposes only and does not constitute an endorsement or promotion. I have no affiliation with Vellum and receive no compensation for its inclusion.

[Submit an Article](#)

Vellum

Identifying the appropriate prompt can pose a challenge. The process begins with a basic prompt, which is then refined through multiple iterations – a task that can be difficult to oversee. Spreadsheets might be adequate for a few experiments, but they falter when faced with the demand for a multitude of iterations. So, the question arises: can we efficiently monitor our prompts? What if we could seamlessly share our prompt experiments with others? Introducing Vellum – answer to these questions

Vellum provides a suite of tools designed for prompt engineering, semantic search, version control, quantitative testing, and performance monitoring to help develop production-grade LLM applications. In the context of prompt engineering, it enables testing and evaluating a given prompt across all major LLMs available in the market. It also facilitates collaboration on prompts.



Screenshot of Vellum sandbox (Screenshot by author)

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

The image above is a representative screenshot sandbox. Vellum enables easy switching between models and allows for effortless adjustment of parameters. It also offers the advantage of later quantitative assessment across four evaluation match, regex match, semantic similarity, and we

Vellum – Developer platform for LLM applica

Summary

This article begins by introducing prompts and then delves into prompt engineering for arithmetic reasoning problems. We explored various prompt engineering techniques, including zero-shot, few-shot, few-shot chain-of-thought, zero-shot chain-of-thought, program-aided language model, and tree-of-thought.

Later, we learned some best practices for crafting better prompts. In general, it's recommended to be specific, provide clear details, refine the prompt continuously through iteration, format it properly with clear separations, and more. **not don'ts.**

Finally, to address the challenge of tracking and managing prompts, as well as sharing them among team members, we investigated the Vellum tool.

If you like this article, make sure to follow me [here](#) to connect and reach out to me via [LinkedIn](#) and [X \(Twitter\)](#).

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

WRITTEN BY

Kaustubh Bhavsar[See all from Kaustubh Bhavsar](#)

[Large Language Models](#)[Llm](#)[NLP](#)[Prompt](#)[Prompt Engineering](#)**Share This Article**

Towards Data Science is a community publication where you can publish your insights to reach our global audience and earn money through the TDS Author Payment Program.

[Write for TDS](#)[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

Related Articles

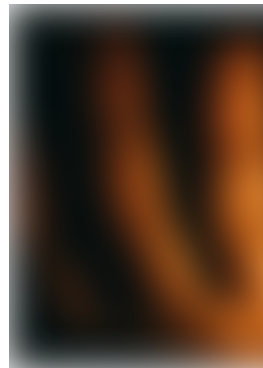
ARTIFICIAL INTELLIGENCE

What Do Large Language Models “Understand”?

A deep dive on the meaning of understanding and how it applies to LLMs

Tarik Dzekman

August 21, 2024 31 min read



NATURAL LANGUAGE PROCESSING

Check Your Bias

Symbolic Engines
Results - A Personal

Veronica Villa

March 17, 2022 9 min read



CINEMA

Evaluating Cinematic Dialogue - Which syntactic and semantic features are predictive of genre?

This article explores the relationship between a movie's dialogue and its genre, leveraging domain-driven data...

Christabelle Pabalan

January 20, 2024 18 min read



NATURAL LANGUAGE PROCESSING

Using NLP and Text Analytics to Cluster Political Texts

NLTK, and scipy on text from Project Gutenberg

Nima Beheshti

January 11, 2022 8 min read



MACHINE LEARNING

Large Language Models into U and Sentence

Discover how mir generates data ai performance on : tasks

Vyacheslav Efimov
December 12, 2023

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

NATURAL LANGUAGE

Linguistic Fin Python

Attributing autho heatmaps

Lee Vaughan

August 1, 2023 10

[Submit an Article](#)



ARTIFICIAL INTELLIGENCE

Making Sense of the Promise (and Risks) of Large Language Models

Our weekly selection of must-read Editors' Picks and original features

TDS Editors

April 27, 2023 4 min read

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in



Submit an Article

Your home for data science and AI. The world's leading publication for data analytics, data engineering, machine learning, and artificial intelligence professionals.

© Insight Media Group, LLC 2025

Subscribe to Our Newsletter

WRITE FOR TDS · ABOUT · ADVERTISE · PRIVACY POLICY

COOKIES SETTINGS